IPL Best Batsman / Best Bowler Prediction

Goal

Build a machine learning pipeline to predict:

- the **best batsman** of each IPL match
- the **best bowler** of each IPL match

The project will use only methods covered in class and will later include a betting simulation based on model probabilities.

Current milestone

In this notebook section, I will:

1. load and inspect the IPL dataset
2. identify the important columns
3. check whether the data is ball-by-ball or already aggregated
4. create player-match level datasets for batsmen and bowlers

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 200)
pd.set_option("display.width", 200)
pd.set_option("display.max_colwidth", 120)
```

Load dataset

The dataset is loaded from CSV and inspected at a high level.

```
file_path = "/content/IPL_compressed.csv"
df = pd.read_csv(file_path)
print(df.shape)
df.head()
```

```
/tmp/ipykernel_7487/219054358.py:2: DtypeWarning: Columns (0,1,2,4,5,6,7,8,9,
  df = pd.read_csv(file_path)
(278212, 64)
```

	Unnamed: 0	match_id	dt	match_type	event_name	innings	batting_tea
0	131970	335982	20080418	T20	0	1	
1	131971	335982	20080418	T20	0	1	
2	131972	335982	20080418	T20	0	1	
3	131973	335982	20080418	T20	0	1	
4	131974	335982	20080418	T20	0	1	

Basic inspection

Check columns, data types, missing values, and a random sample of rows.

```
print("Number of columns:", len(df.columns))
print("\nColumns:\n")
print(df.columns.tolist())
```

```
Number of columns: 64

Columns:

['Unnamed: 0', 'match_id', 'dt', 'match_type', 'event_name', 'innings', 'batt
```

```
df.dtypes.sort_index()
```

	0
Unnamed: 0	object
ball	object
ball_no	object
balls_faced	object
balls_per_over	float64
bat_pos	object
batter	object
batter_balls	float64
batter_runs	float64
batting_partners	object
batting_team	object
bowler	object
bowler_wicket	float64
bowling_team	object
city	object
day	float64
dt	object
event_match_no	object
event_name	object
extra_type	object
fielders	object
gender	object
innings	object
match_id	object
match_number	object
match_type	object
match_won_by	float64
method	object
month	float64
new_batter	object
next_batter	object
non_striker	object

```
missing = df.isna().sum().sort_values(ascending=False)
missing[missing > 0].head(50)
```

overs	float64
player_of_match	object
player_out	object
result_type	object
review_batter	object
review_decision	object
runs_batter	object
runs_bowler	object
runs_extras	object
runs_not_boundary	object
runs_target	object
runs_total	object
season	object
stage	object
striker_out	object
superover_winner	object
team_balls	float64
team_reviewed	object
team_runs	float64
team_type	object
team_wicket	float64
toss_decision	object
toss_winner	float64
umpire	object
umpires_call	object
valid_ball	object
venue	float64
wicket_kind	object
win_outcome	object
year	float64

dtype: object

	0
team_reviewed	277340
review_decision	277340
umpire	277340
review_batter	277339
method	274322
superover_winner	274316
result_type	273510
fielders	268198
new_batter	264891
next_batter	264891
player_out	264388
wicket_kind	264388
extra_type	263078
runs_target	144308
win_outcome	4709
overs	7
match_won_by	7
player_of_match	7
season	7
month	7
city	7
match_number	7
year	7
team_type	7
gender	7

```
df.sample(5, random_state=42)
```

	Unnamed: 0	match_id	dt	match_type	event_name	innings	battin
183109	47631	1216513	20201004	T20	0	1	
61230	193200	548317	20120411	T20	0	1	
139022	2424	1082601	20170413	T20	0	1	
247745	111147	1422174	20240403	T20	0	2	
45966	177936	501216	20110419	T20	0	2	

team_balls	7
team_runs	7
batter_runs	7
team_wicket	7

Candidate key columns

Try to detect likely columns for match id, date, batter, bowler, teams, and ball outcomes. These will be reviewed before aggregation.

```
cols = df.columns.tolist()
lower_map = {c.lower(): c for c in cols}

def find_candidates(keywords):
    out = []
    for c in cols:
        c_low = c.lower()
        if any(k in c_low for k in keywords):
            out.append(c)
    return out

candidate_map = {
    "match_id": find_candidates(["match_id", "id", "match"]),
    "date": find_candidates(["date"]),
    "season": find_candidates(["season", "year"]),
    "batter": find_candidates(["batter", "batsman", "striker"]),
    "bowler": find_candidates(["bowler"]),
    "batting_team": find_candidates(["batting_team", "bat_team"]),
    "bowling_team": find_candidates(["bowling_team", "bowl_team"]),
    "venue": find_candidates(["venue", "ground", "stadium"]),
    "over": find_candidates(["over"]),
    "ball": find_candidates(["ball"]),
    "runs_batter": find_candidates(["batsman_runs", "batter_runs", "runs_off
    "runs_total": find_candidates(["total_runs", "runs_total"]),
    "extras": find_candidates(["extra", "extras"]),
    "wicket_flag": find_candidates(["is_wicket", "wicket"]),
    "dismissed_player": find_candidates(["player_dismissed", "dismissed", "c
    "fielder": find_candidates(["fielder"]),
    "inning": find_candidates(["inning", "innings"])
}

for k, v in candidate_map.items():
    print(f"{k}: {v}")
```

```
match_id: ['match_id', 'match_type', 'valid_ball', 'player_of_match', 'match_
date: []
season: ['year', 'season']
batter: ['batter', 'runs_batter', 'non_striker', 'non_striker_pos', 'review_b
bowler: ['bowler', 'runs_bowler', 'bowler_wicket']
batting_team: ['batting_team']
bowling_team: ['bowling_team']
venue: ['venue']
over: ['over', 'superover_winner', 'balls_per_over', 'overs']
ball: ['ball', 'ball_no', 'balls_faced', 'valid_ball', 'balls_per_over', 'tea
runs_batter: ['runs_batter', 'batter_runs']
runs_total: ['runs_total']
extras: ['runs_extras', 'extra_type']
wicket_flag: ['wicket_kind', 'team_wicket', 'bowler_wicket']
dismissed_player: []
```

```
fielder: ['fielders']
inning: ['innings']
```

Set the working column names

The automatic scan showed that this dataset uses different names than the initial template. The variables below are updated to match the real IPL dataset columns.

```
MATCH_COL = "match_id"
DATE_COL = "year"
SEASON_COL = "season"

BATTER_COL = "batter"
BOWLER_COL = "bowler"

BATTING_TEAM_COL = "batting_team"
BOWLING_TEAM_COL = "bowling_team"

VENUE_COL = "venue"

OVER_COL = "over"
BALL_COL = "ball"

BATTER_RUNS_COL = "runs_batter"
TOTAL_RUNS_COL = "runs_total"
EXTRAS_COL = "runs_extras"

WICKET_COL = "bowler_wicket"
DISMISSED_PLAYER_COL = None

INNING_COL = "innings"
```

Validate selected columns

Before moving forward, confirm that the required columns exist in the dataset.

```
required_cols = [
    MATCH_COL, DATE_COL, BATTER_COL, BOWLER_COL,
    BATTING_TEAM_COL, BOWLING_TEAM_COL, VENUE_COL,
    BATTER_RUNS_COL, TOTAL_RUNS_COL
]

missing_required = [c for c in required_cols if c not in df.columns]
print("Missing required columns:", missing_required)
```

```
Missing required columns: []
```

Clean the time column and sort chronologically

The dataset does not include a full match date, so the `year` column will be used as the time reference.

This column must be cleaned before sorting because some rows may contain missing or non-numeric values.

```
df[DATE_COL] = pd.to_numeric(df[DATE_COL], errors="coerce")

print("Missing values in time column before cleaning:", df[DATE_COL].isna().
display(df[df[DATE_COL].isna()][[MATCH_COL, DATE_COL]].head(10))

df = df.dropna(subset=[DATE_COL]).copy()
df[DATE_COL] = df[DATE_COL].astype(int)

df = df.sort_values([DATE_COL, MATCH_COL]).reset_index(drop=True)

print("Shape after dropping missing years:", df.shape)
print(df[[DATE_COL, MATCH_COL]].head())
print(df[[DATE_COL, MATCH_COL]].tail())
```

```
Missing values in time column before cleaning: 7
```

	match_id	year
278205	NaN	NaN
278206	NaN	NaN
278207	NaN	NaN
278208	NaN	NaN
278209	NaN	NaN
278210	NaN	NaN
278211	Mohali"|2=Feroz Shah Kotla|3=Wankhede Stadium|4=Eden Gardens|5=Sawai Mansingh Stadium|6="Rajiv Gandhi International...	NaN

```
Shape after dropping missing years: (278205, 64)
   year match_id
0  2008   335982
1  2008   335982
2  2008   335982
3  2008   335982
4  2008   335982
        year match_id
278200  2025  1485779
278201  2025  1485779
278202  2025  1485779
278203  2025  1485779
278204  2025  1485779
```

Check row granularity

This inspection helps confirm whether the dataset is ball-by-ball and whether aggregation is needed.

```
granularity_cols = [
    c for c in [
        MATCH_COL, INNING_COL, OVER_COL, BALL_COL,
        BATTER_COL, BOWLER_COL, BATTER_RUNS_COL, TOTAL_RUNS_COL
    ]
    if c in df.columns
]

df[granularity_cols].head(20)
```

	match_id	innings	over	ball	batter	bowler	runs_batter	runs_total
0	335982	1	0	1	SC Ganguly	P Kumar	0	1
1	335982	1	0	2	BB McCullum	P Kumar	0	0
2	335982	1	0	3	BB McCullum	P Kumar	0	1
3	335982	1	0	3	BB McCullum	P Kumar	0	0
4	335982	1	0	4	BB McCullum	P Kumar	0	0
5	335982	1	0	5	BB McCullum	P Kumar	0	0
6	335982	1	0	6	BB McCullum	P Kumar	0	1
7	335982	1	1	1	BB McCullum	Z Khan	0	0
8	335982	1	1	2	BB McCullum	Z Khan	4	4
9	335982	1	1	3	BB McCullum	Z Khan	4	4
10	335982	1	1	4	BB McCullum	Z Khan	6	6
11	335982	1	1	5	BB McCullum	Z Khan	4	4
12	335982	1	1	6	BB McCullum	Z Khan	0	0

```
print("Rows:", len(df))
print("Matches:", df[MATCH_COL].nunique())
print("Batters:", df[BATTER_COL].nunique())
print("Bowlers:", df[BOWLER_COL].nunique())
```

```
Rows: 278205
Matches: 1170
Batters: 703
Bowlers: 550
```

Build batsman match-level table

Each row in the new table should represent one batter in one match.

```
group_batter = [MATCH_COL, DATE_COL, BATTER_COL, BATTING_TEAM_COL, BOWLING_T
if SEASON_COL in df.columns:
    group_batter.append(SEASON_COL)

batsman_match = (
    df.groupby(group_batter, dropna=False)
      .agg(
          innings_balls=(BATTER_COL, "size"),
          runs=(BATTER_RUNS_COL, "sum"),
          fours=(BATTER_RUNS_COL, lambda x: (pd.to_numeric(x, errors="coerce
          sixes=(BATTER_RUNS_COL, lambda x: (pd.to_numeric(x, errors="coerce
          total_runs_including_extras=(TOTAL_RUNS_COL, "sum")
      )
      .reset_index()
)

batsman_match["runs"] = pd.to_numeric(batsman_match["runs"], errors="coerce"
batsman_match["total_runs_including_extras"] = pd.to_numeric(
    batsman_match["total_runs_including_extras"], errors="coerce"
)

batsman_match["balls_faced"] = pd.to_numeric(batsman_match["innings_balls"],
batsman_match["strike_rate"] = np.where(
    batsman_match["balls_faced"] > 0,
    100 * batsman_match["runs"] / batsman_match["balls_faced"],
    0
)

batsman_match.head()
```

	match_id	year	batter	batting_team	bowling_team	venue	season	innir
0	335982	2008	AA Noffke	1	1	0.0	2007/08	
1	335982	2008	B Akhil	1	1	0.0	2007/08	
2	335982	2008	BB McCullum	0	0	0.0	2007/08	
3	335982	2008	CL White	1	1	0.0	2007/08	

Build bowler match-level table

Each row in the new table should represent one bowler in one match.

```
group_bowler = [MATCH_COL, DATE_COL, BOWLER_COL, BOWLING_TEAM_COL, BATTING_T
if SEASON_COL in df.columns:
    group_bowler.append(SEASON_COL)

bowler_match = (
    df.groupby(group_bowler, dropna=False)
      .agg(
          balls_bowled=(BOWLER_COL, "size"),
          runs_conceded=(TOTAL_RUNS_COL, "sum"),
          wickets=(WICKET_COL, "sum")
      )
      .reset_index()
)

bowler_match["runs_conceded"] = pd.to_numeric(bowler_match["runs_conceded"],
bowler_match["wickets"] = pd.to_numeric(bowler_match["wickets"], errors="coe
bowler_match["balls_bowled"] = pd.to_numeric(bowler_match["balls_bowled"], e

bowler_match["overs_bowled"] = bowler_match["balls_bowled"] / 6
bowler_match["economy"] = np.where(
    bowler_match["overs_bowled"] > 0,
    bowler_match["runs_conceded"] / bowler_match["overs_bowled"],
    0
)

bowler_match.head()
```

	match_id	year	bowler	bowling_team	batting_team	venue	season	balls_b
0	335982	2008	AA Noffke	0	0	0.0	2007/08	
1	335982	2008	AB Agarkar	1	1	0.0	2007/08	
2	335982	2008	AB Dinda	1	1	0.0	2007/08	

Check aggregated datasets

Verify that both tables now contain one row per player per match.

```
print("Batsman match table shape:", batsman_match.shape)
print("Bowler match table shape:", bowler_match.shape)

print("\nDuplicate batter-match rows:",
```

```
      batsman_match.duplicated(subset=[MATCH_COL, BATTER_COL]).sum())

print("Duplicate bowler-match rows:",
      bowler_match.duplicated(subset=[MATCH_COL, BOWLER_COL]).sum())
```

```
Batsman match table shape: (17650, 14)
Bowler match table shape: (13870, 12)

Duplicate batter-match rows: 10
Duplicate bowler-match rows: 19
```

```
batsman_match.sample(10, random_state=42)
```

	match_id	year	batter	batting_team	bowling_team	venue	season
3502	501255	2011	KB Arun Karthik	1	1	0.0	2011
6850	734047	2014	F du Plessis	2	2	3.0	2014
4501	548358	2012	WD Parnell	9	5	26.0	2012
5290	598026	2013	AM Rahane	4	3	7.0	2013
10425	1136620	2018	YK Pathan	10	3	3.0	2018
5282	598025	2013	Azhar Mahmood	3	9	1.0	2013
5066	598011	2013	Yuvraj Singh	9	6	3.0	2013
9985	1136591	2018	C de Grandhomme	1	6	36.0	2018
5216	598020	2013	SK Raina	2	4	2.0	2013
17419	1473497	2025	SO Hetmyer	4	15	54.0	2025

```
bowler_match.sample(10, random_state=42)
```

	match_id	year	bowler	bowling_team	batting_team	venue	season	ba
2334	501214	2011	SR Watson	5	0	4.0	2011	
1078	392214	2009	L Balaji	3	3	12.0	2009	
6647	980983	2016	BB Sran	10	5	6.0	2016	
6967	1082598	2017	P Negi	0	3	24.0	2017	
5044	733977	2014	DW Steyn	10	1	0.0	2014	
360	336011	2008	YK Pathan	5	7	5.0	2007/08	
977	392206	2009	L Balaji	3	5	14.0	2009	
1650	419126	2010	WPUJC Vaas	7	4	18.0	2009/10	

Create scoring formulas

These custom scores will define the labels for the best batsman and best bowler of each match.

```
batsman_match["batsman_score"] = (
    batsman_match["runs"]
    + 0.3 * batsman_match["strike_rate"]
    + 2 * batsman_match["fours"]
    + 3 * batsman_match["sixes"]
)

bowler_match["bowler_score"] = (
    25 * bowler_match["wickets"]
    - 2 * bowler_match["economy"]
    - 0.5 * bowler_match["runs_conceded"]
)
```

Create labels

For each match:

- the batter with the highest batsman score receives label `1`
- the bowler with the highest bowler score receives label `1`
- all other players receive label `0`

```
batsman_match["best_batsman_label"] = (
    batsman_match.groupby(MATCH_COL)["batsman_score"]
    .transform(lambda x: (x == x.max()).astype(int))
```

```
)

bowler_match["best_bowler_label"] = (
    bowler_match.groupby(MATCH_COL)["bowler_score"]
    .transform(lambda x: (x == x.max()).astype(int))
)
```

```
print(batsman_match["best_batsman_label"].value_counts())
print(bowler_match["best_bowler_label"].value_counts())
```

```
best_batsman_label
0    16479
1     1171
Name: count, dtype: int64
best_bowler_label
0    12696
1     1174
Name: count, dtype: int64
```

Quick sanity checks

Inspect a few matches to verify that the scoring formulas and labels look reasonable.

```
sample_match_ids = batsman_match[MATCH_COL].drop_duplicates().sample(3, rand

for match_id in sample_match_ids:
    print(f"\nMATCH {match_id} — BATSMEN")
    display(
        batsman_match.loc[
            batsman_match[MATCH_COL] == match_id,
            [MATCH_COL, BATTER_COL, "runs", "balls_faced", "strike_rate", "f
        ]
        .sort_values("batsman_score", ascending=False)
        .head(10)
    )
```

```
MATCH 1082641 — BATSMEN
```

	match_id	batter	runs	balls_faced	strike_rate	fours	sixes	batsı
9377	1082641	WP Saha	93.0	59	157.627119	11	3	1

```
for match_id in sample_match_ids:
    print(f"\nMATCH {match_id} — BOWLERS")
    display(
        bowler_match.loc[
            bowler_match[MATCH_COL] == match_id,
            [MATCH_COL, BOWLER_COL, "balls_bowled", "overs_bowled", "runs_co
        ]
        .sort_values("bowler_score", ascending=False)
        .head(10)
    )
```

9372	1082641	MJ Guptill	36.0	18	200.000000	5	1	1
9367	1082641	HH Pandya	30.0	14	214.285714	0	4	1
9374	1082641	PA Patel	38.0	24	158.333333	7	0	
9376	1082641	SE Marsh	25.0	17	147.058824	0	2	
9365	1082641	AR Patel	19.0	13	146.153846	0	1	

```
MATCH 501244 — BATSMEN
```

	match_id	batter	runs	balls_faced	strike_rate	fours	sixes	batsı
3339	501244	CH Gayle	107.0	49	218.367347	10	9	2
3335	501244	AB de Villiers	27.0	14	192.857143	2	2	
3343	501244	P Kumar	15.0	8	187.500000	2	1	
3351	501244	TM Dilshan	16.0	9	177.777778	4	0	
3334	501244	A Mithun	6.0	3	200.000000	1	0	
3340	501244	KD Karthik	20.0	15	133.333333	3	0	
3344	501244	PC Valthaty	21.0	18	116.666667	4	0	
3349	501244	SS Tiwary	13.0	9	144.444444	2	0	
3352	501244	V Kohli	27.0	32	84.375000	4	0	
3346	501244	R McLaren	28.0	33	84.848485	1	0	

```
MATCH 1136603 — BATSMEN
```

	match_id	batter	runs	balls_faced	strike_rate	fours	sixes	bat
10162	1136603	JC Buttler	95.0	60	158.333333	11	2	

```
MATCH 1082641 — BOWLERS
```

	match_id	bowler	balls_bowled	overs_bowled	runs_conceded	wic
7465	1082641	GJ Maxwell	12	2.000000	9.0	
7469	1082641	JJ Bumrah	24	4.000000	25.0	
7474	1082641	R Tewatia	12	2.000000	17.0	
7473	1082641	MM Sharma	24	4.000000	57.0	
7464	1082641	AR Patel	18	3.000000	28.0	
7470	1082641	KV Sharma	18	3.000000	32.0	
7472	1082641	MJ McClenaghan	28	4.666667	54.0	
7468	1082641	I Sharma	19	3.166667	30.0	
7466	1082641	HH Pandya	13	2.166667	29.0	

```
MATCH 501244 — BOWLERS
```

	match_id	bowler	balls_bowled	overs_bowled	runs_conceded	wickets
2692	501244	S Aravind	24	4.000000	14.0	4.0
2685	501244	CH Gayle	24	4.000000	21.0	3.0
2691	501244	RJ Harris	24	4.000000	42.0	3.0
2689	501244	PP Chawla	24	4.000000	37.0	2.0
2686	501244	DL Vettori	24	4.000000	14.0	1.0
2690	501244	R McLaren	25	4.166667	41.0	1.0
2693	501244	V Kohli	7	1.166667	7.0	0.0
		...				
2694	501244	Z Khan	27	4.500000	35.0	0.0

```
MATCH 1136603 — BOWLERS
```

	match_id	bowler	balls_bowled	overs_bowled	runs_conceded	wicket

Save processed tables

The match-level batsman and bowler datasets are saved for the next stage of the project.

```
batsman_match.to_csv("/content/batsman_match_level.csv", index=False)
bowler_match.to_csv("/content/bowler_match_level.csv", index=False)
```

Outcome of this section

At this point, the raw IPL dataset has been transformed into:

- a batsman match-level table
- a bowler match-level table

Both tables now include:

- aggregated per-match player statistics
- custom performance scores
- binary labels for the best player in each match

The next step will be to create **pre-match historical features** so that the models can make realistic betting-style predictions using only past information.

Create pre-match historical features

To make realistic predictions, features must be computed using only past matches.

Rolling averages will be used to capture recent player form.

```
batsman_match = batsman_match.sort_values([BATTER_COL, DATE_COL])
bowler_match = bowler_match.sort_values([BOWLER_COL, DATE_COL])
```

Batsman rolling features

These features capture recent batting performance using previous matches only.

```
batsman_match["runs_last_3"] = (
    batsman_match.groupby(BATTER_COL)["runs"]
    .shift(1)
    .rolling(3)
    .mean()
)

batsman_match["runs_last_5"] = (
    batsman_match.groupby(BATTER_COL)["runs"]
    .shift(1)
    .rolling(5)
    .mean()
)

batsman_match["sr_last_5"] = (
    batsman_match.groupby(BATTER_COL)["strike_rate"]
    .shift(1)
    .rolling(5)
    .mean()
)

batsman_match["fours_last_5"] = (
    batsman_match.groupby(BATTER_COL)["fours"]
    .shift(1)
    .rolling(5)
    .mean()
)

batsman_match["sixes_last_5"] = (
    batsman_match.groupby(BATTER_COL)["sixes"]
    .shift(1)
    .rolling(5)
    .mean()
)
```

Bowler rolling features

These features capture recent bowling performance using previous matches only.

```
bowler_match["wickets_last_3"] = (
    bowler_match.groupby(BOWLER_COL)["wickets"]
    .shift(1)
    .rolling(3)
    .mean()
)

bowler_match["wickets_last_5"] = (
    bowler_match.groupby(BOWLER_COL)["wickets"]
    .shift(1)
    .rolling(5)
    .mean()
)
```

```
bowler_match["eco_last_5"] = (
    bowler_match.groupby(BOWLER_COL)["economy"]
    .shift(1)
    .rolling(5)
    .mean()
)

bowler_match["runs_conceded_last_5"] = (
    bowler_match.groupby(BOWLER_COL)["runs_conceded"]
    .shift(1)
    .rolling(5)
    .mean()
)
```

Remove rows without history

The first matches of each player do not have enough past data and must be removed.

```
batsman_model_df = batsman_match.dropna().copy()
bowler_model_df = bowler_match.dropna().copy()

print("Batsman dataset:", batsman_model_df.shape)
print("Bowler dataset:", bowler_model_df.shape)
```

```
Batsman dataset: (14844, 21)
Bowler dataset: (11615, 18)
```

```
print(batsman_model_df.head())
print(bowler_model_df.head())
```

```
     match_id  year          batter batting_team bowling_team  venue season
4872   598000  2013  A Ashish Reddy           10            9    6.0   2013
4939   598004  2013  A Ashish Reddy           10            0    6.0   2013
5033   598010  2013  A Ashish Reddy           10            4    2.0   2013
5082   598013  2013  A Ashish Reddy           10            1    4.0   2013
5166   598018  2013  A Ashish Reddy           10            9   27.0   2013

      best_batsman_label  runs_last_3  runs_last_5   sr_last_5  fours_last_5
4872                   0     7.333333          7.0  126.000000           0.6
4939                   0     7.000000          6.4  141.000000           0.8
5033                   0     8.333333          8.6  144.333333           0.8
5082                   0    12.333333         10.2  159.888889           1.0
5166                   0    11.333333          9.0  125.888889           0.6
     match_id  year          bowler bowling_team batting_team  venue season
3580   548356  2012  A Ashish Reddy            7            1    0.0   2012
3616   548359  2012  A Ashish Reddy            7            3    6.0   2012
3777   548373  2012  A Ashish Reddy            7            4    6.0   2012
3811   548376  2012  A Ashish Reddy            7            1    6.0   2012
3905   598000  2013  A Ashish Reddy           10            9    6.0   2013

      wickets_last_5  eco_last_5  runs_conceded_last_5
3580             1.0    8.940833                  24.0
3616             1.2    7.768833                  25.4
```

```
3777            1.2    8.088833                 27.0
3811            1.0    8.845976                 28.2
3905            1.4    8.024923                 26.8
```

Time-based train/test split

The dataset is split chronologically to avoid using future data during training.

```
train_batsman = batsman_model_df[batsman_model_df[DATE_COL] <= 2019]
test_batsman = batsman_model_df[batsman_model_df[DATE_COL] > 2019]

train_bowler = bowler_model_df[bowler_model_df[DATE_COL] <= 2019]
test_bowler = bowler_model_df[bowler_model_df[DATE_COL] > 2019]

print("Batsman:", train_batsman.shape, test_batsman.shape)
print("Bowler:", train_bowler.shape, test_bowler.shape)
```

```
Batsman: (9248, 21) (5596, 21)
Bowler: (7324, 18) (4291, 18)
```

Define feature sets

These columns will be used as model inputs for predicting the best batsman and best bowler.

```
batsman_features = [
    "runs_last_3",
    "runs_last_5",
    "sr_last_5",
    "fours_last_5",
    "sixes_last_5"
]

bowler_features = [
    "wickets_last_3",
    "wickets_last_5",
    "eco_last_5",
    "runs_conceded_last_5"
]
```

Clean feature matrices

Ensure all selected features are numeric and free from missing or infinite values before training.

```
def clean_feature_block(df, feature_cols):
    df = df.copy()
    df[feature_cols] = df[feature_cols].apply(pd.to_numeric, errors="coerce"
    df[feature_cols] = df[feature_cols].replace([np.inf, -np.inf], np.nan)
    df[feature_cols] = df[feature_cols].fillna(0)
    df[feature_cols] = df[feature_cols].clip(lower=-1e6, upper=1e6)
    return df

train_batsman = clean_feature_block(train_batsman, batsman_features)
test_batsman = clean_feature_block(test_batsman, batsman_features)

train_bowler = clean_feature_block(train_bowler, bowler_features)
test_bowler = clean_feature_block(test_bowler, bowler_features)
```

Build training and test matrices

Create the feature matrices `X` and target vectors `y` for both tasks.

```
X_train_bat = train_batsman[batsman_features].astype("float64")
y_train_bat = train_batsman["best_batsman_label"]

X_test_bat = test_batsman[batsman_features].astype("float64")
y_test_bat = test_batsman["best_batsman_label"]

X_train_bowl = train_bowler[bowler_features].astype("float64")
y_train_bowl = train_bowler["best_bowler_label"]

X_test_bowl = test_bowler[bowler_features].astype("float64")
y_test_bowl = test_bowler["best_bowler_label"]
```

Quick validation checks

Confirm that the feature matrices do not contain invalid values before training.

```
print("Batsman train inf:", np.isinf(X_train_bat.to_numpy()).sum())
print("Batsman test inf:", np.isinf(X_test_bat.to_numpy()).sum())
print("Bowler train inf:", np.isinf(X_train_bowl.to_numpy()).sum())
print("Bowler test inf:", np.isinf(X_test_bowl.to_numpy()).sum())

print("Batsman train nan:", np.isnan(X_train_bat.to_numpy()).sum())
print("Batsman test nan:", np.isnan(X_test_bat.to_numpy()).sum())
print("Bowler train nan:", np.isnan(X_train_bowl.to_numpy()).sum())
print("Bowler test nan:", np.isnan(X_test_bowl.to_numpy()).sum())
```

```
Batsman train inf: 0
Batsman test inf: 0
Bowler train inf: 0
Bowler test inf: 0
Batsman train nan: 0
Batsman test nan: 0
```